

计算机视觉任务与模型概览

一、四大核心任务

计算机视觉任务与模型概览

一、四大核心任务

计算机视觉四大基础任务构成从粗粒度到细粒度的感知体系：

任务 核心能力 典型描述 输出形式

图像分类 全局特征判断图像所属类别 " 这张图片是猫还是狗 " 类别标签

物体检测 分类 + 空间定位，输出边界框坐标与类别标签 找到目标位置并识别 边界框 + 类别

语义分割 像素级分类，区分不同语义区域但忽略个体差异 标注每个像素的类别 像素级标签图

实例分割 在语义分割基础上区分同类不同个体 最精细的视觉感知 像素级标签 + 实例 I D

技术演进路径：图像级理解（分类）→ 空间定位（检测）→ 像素级理解（分割）→ 个体级区分（实例分割）

任务粒度对比图

图像分类： [整张图 → " 猫 "] 粒度：图像级

物体检测： [猫（框 1），狗（框 2），椅子（框 3）] 粒度：区域级

语义分割： [每个像素标注：猫 / 狗 / 背景 / 地面] 粒度：像素级（不区分类内个体）

实例分割： [猫 1（像素集 A），猫 2（像素集 B），狗（像素集 C）] 粒度：像素级 + 实例级

- - -

二、图像分类详解

2 . 1 核心架构

基于 C N N 的图像分类包含特征提取和分类决策两个阶段：

输入图像 → [特征提取层] → [分类层] → 类别概率

↑ ↑

卷积 + 池化 + 激活 全连接 + S o f t m a x

分类模型的数学表达：

$$P(y=c|x) = \text{Softmax}(f(\theta(x)))c = \frac{\exp(f(\theta(x))c)}{\sum_{k=1}^K \exp(f(\theta(x))k)}$$

其中 $z = f(\theta(x))$ 是模型对输入 x 的 logits 输出, K 是类别数

2.2 数据增强完整技术

数据增强是防止过拟合、提升模型泛化能力的关键手段。以下是常用数据增强技术的完整列表：

几何变换

```
import torchvision.transforms as T
```

===== 基础几何变换 =====

```
geometric_transforms = T.Compose([
```

```
# 随机裁剪：从原图中随机裁剪指定大小的区域
```

```
# 训练时用RandomResizedCrop，推理时用CenterCrop
```

```
T.RandomResizedCrop(size=224, scale=(0.08, 1.0),
```

```
# 随机水平翻转：50%概率水平镜像
```

```
T.RandomHorizontalFlip(p=0.5),
```

```
# 随机垂直翻转：10%概率垂直镜像（适用于卫星图等）
```

```
T.RandomVerticalFlip(p=0.1),
```

```
# 随机旋转：在  $[-15^\circ, +15^\circ]$  范围内随机旋转
```

```
T.RandomRotation(degrees=15),
```

```
# 随机仿射变换：包含旋转、平移、缩放、剪切
```

```
T.RandomAffine(
```

```
degrees=10, # 旋转角度范围
```

```
translate=(0.1, 0.1), # 平移比例
```

```
scale=(0.9, 1.1), # 缩放范围
```

```
shear=5 # 剪切角度
```

```
),
```

```
# 随机透视变换：模拟不同拍摄角度
```

```
T.RandomPerspective(distortionscale=0.2, p=0.5),
```

随机擦除：模拟遮挡

```
T.RandomErasing(p=0.5, scale=(0.02, 0.2), ratio=[
```

==== 高级：Mixup =====

Mixup：将两张图像按比例混合，标签也对应混合

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j$$
$$\tilde{y} = \lambda y_i + (1 - \lambda) y_j$$

$\lambda \sim \text{Beta}(\alpha, \alpha)$ ， α 通常取0.2或1.0

```
import torch
```

```
import numpy as np
```

```
def mixupdata(x, y, alpha=0.2):
```

```
    """Mixup 数据增强"""
```

```
    if alpha > 0:
```

```
        lam = np.random.beta(alpha, alpha)
```

```
    else:
```

```
        lam = 1.0
```

```
    batchsize = x.size(0)
```

```
    index = torch.randperm(batchsize) # 随机打乱索引
```

混合图像

```
mixedx = lam x + (1 - lam) x[index]
```

混合标签

```
ya, yb = y, y[index]
```

```
return mixedx, ya, yb, lam
```

```
def mixupcriterion(criterion, pred, ya, yb, lam):
```

```
    """Mixup 损失函数"""
```

```
    return lam criterion(pred, ya) + (1 - lam) criterion
```

==== 高级：CutMix =====

CutMix：从一张图裁剪矩形区域粘贴到另一张图上

```

def cutmixdata(x, y, alpha=1.0):
    """CutMix 数据增强"""
    lam = np.random.beta(alpha, alpha)
    batchsize = x.size(0)
    index = torch.randperm(batchsize)

    # 生成随机裁剪区域
    W, H = x.size(2), x.size(3)
    cutratio = np.sqrt(1.0 - lam)
    cutw = int(W * cutratio)
    cuth = int(H * cutratio)

    cx = np.random.randint(W)
    cy = np.random.randint(H)

    x1 = np.clip(cx - cutw // 2, 0, W)
    y1 = np.clip(cy - cuth // 2, 0, H)
    x2 = np.clip(cx + cutw // 2, 0, W)
    y2 = np.clip(cy + cuth // 2, 0, H)

    # 粘贴区域
    mixedx = x.clone()
    mixedx[:, :, x1:x2, y1:y2] = x[index, :, x1:x2, y1:y2]

    # 调整lambda (基于面积比)
    lam = 1 - (x2 - x1) * (y2 - y1) / (W * H)

    return mixedx, y, y[index], lam

```

颜色变换

```

colortransforms = T.Compose([
    # 颜色抖动：随机改变亮度、对比度、饱和度、色调
    T.ColorJitter(
        brightness=0.2, # 亮度变化因子
        contrast=0.2, # 对比度变化因子

```

```

s a t u r a t i o n = 0 . 2 ,   #   饱和度变化因子
h u e = 0 . 1   #   色调变化因子 ( 0 - 0 . 5 )
) ,

#   随机灰度化：1 0 % 概率转为灰度图
T . R a n d o m G r a y s c a l e ( p = 0 . 1 ) ,

#   高斯模糊
T . G a u s s i a n B l u r ( k e r n e l s i z e = 5 ,   s i g m a = ( 0 . 1 ,   2 . 0 ) ) ,

#   随机反转
T . R a n d o m I n v e r t ( p = 0 . 1 ) ,

#   随机调整锐度
T . R a n d o m A d j u s t S h a r p n e s s ( s h a r p n e s s f a c t o r = 2 ,   p = 0 . 5 )

#   随机海报化 (减少颜色数)
T . R a n d o m P o s t e r i z e ( b i t s = 2 ,   p = 0 . 1 ) ,

#   随机太阳能化 (反转超过阈值的像素)
T . R a n d o m S o l a r i z e ( t h r e s h o l d = 1 2 8 ,   p = 0 . 1 ) ,
] )

=====   A u t o A u g m e n t   和   R a n d A u g m e n t   =====
A u t o A u g m e n t :   使用搜索算法找到最优增强策略
a u t o a u g m e n t   =   T . A u t o A u g m e n t ( p o l i c y = T . A u t o A u g m e n t P

R a n d A u g m e n t :   简化版本, 只需指定增强数量和幅度
r a n d a u g m e n t   =   T . R a n d A u g m e n t ( n u m o p s = 2 ,   m a g n i t u d e = 9

=====   完整训练数据增强流水线   =====
t r a i n t r a n s f o r m   =   T . C o m p o s e ( [
T . R a n d o m R e s i z e d C r o p ( 2 2 4 ,   s c a l e = ( 0 . 0 8 ,   1 . 0 ) ) ,
T . R a n d o m H o r i z o n t a l F l i p ( p = 0 . 5 ) ,
T . T r i v i a l A u g m e n t W i d e ( ) ,   #   自动选择增强策略
T . T o T e n s o r ( ) ,
T . N o r m a l i z e ( m e a n = [ 0 . 4 8 5 ,   0 . 4 5 6 ,   0 . 4 0 6 ] ,   s t d = [ 0 . 2 2

```

```
T.RandomErasing(p=0.25),
])
```

推理时只做必要的预处理

```
val transform = T.Compose([
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.22
])
```

2.3 迁移学习详解

迁移学习是CV中最实用的技术之一，核心思想：在大数据集上预训练的模型已经学到了通用的视觉特征，可以直接迁移到新任务上。

```
import torch
import torch.nn as nn
from torchvision import models
```

```
=====
```

方法1：特征提取（冻结预训练层，只训练分类头）

```
=====
```

适用场景：目标数据集很小（<1000张），且与ImageNet相似

```
model_fe = models.resnet50(pretrained=True)
```

冻结所有参数

```
for param in model_fe.parameters():
    param.requires_grad = False
```

替换分类头

```
numfeatures = model_fe.fc.infeatures # 2048
model_fe.fc = nn.Linear(numfeatures, 10) # 10分类任务
```

只有fc层的参数会被更新

```
print(f"可训练参数： {sum(p.numel() for p in model_fe.pa
```

```
print(f"总参数: {sum(p.numel() for p in model.parameters())}")
```

=====

方法2：微调（Fine-tuning，解冻部分或全部层）

=====

适用场景：目标数据集中等大小，或与ImageNet差异较大

```
model_ft = models.resnet50(pretrained=True)
```

策略A：全部解冻（数据集较大时）

```
for param in model_ft.parameters():
    param.requires_grad = True
```

策略B：只解冻后面的层（数据集较小时）

冻结前面的层（低级特征），只训练后面的层（高级特征）

```
for name, param in model_ft.named_parameters():
    if "layer4" not in name and "fc" not in name:
        param.requires_grad = False
```

替换分类头

```
model_ft.fc = nn.Linear(model_ft.fc.in_features, 100)
```

=====

方法3：差异化学习率

=====

不同层使用不同学习率：浅层小学习率，深层大学习率

```
optimizer = torch.optim.SGD([
    {'params': model_ft.conv1.parameters(), 'lr': 1e-5},
    {'params': model_ft.layer1.parameters(), 'lr': 1e-5},
    {'params': model_ft.layer2.parameters(), 'lr': 5e-5},
    {'params': model_ft.layer3.parameters(), 'lr': 1e-4},
    {'params': model_ft.layer4.parameters(), 'lr': 5e-4},
    {'params': model_ft.fc.parameters(), 'lr': 1e-3}],
    momentum=0.9, weight_decay=1e-4)
```

=====

完整训练脚本

=====

```
def trainwithtransferlearning(
    numclasses=10,
    datadir = './data',
    epochs=20,
    batchsize=32,
    learningrate=1e-3,
    strategy='finetune'
):
    """ 使用迁移学习训练图像分类模型 """

    # 1. 数据准备
    from torchvision import datasets
    traindataset = datasets.ImageFolder(
        f'{datadir}/train', transform=traintransform
    )
    valdataset = datasets.ImageFolder(
        f'{datadir}/val', transform=valtransform
    )

    trainloader = torch.utils.data.DataLoader(
        traindataset, batchsize=batchsize, shuffle=True,
    )
    valloader = torch.utils.data.DataLoader(
        valdataset, batchsize=batchsize, shuffle=False,
    )

    # 2. 模型准备
    model = models.resnet50(pretrained=True)

    if strategy == 'featureextract':
        for param in model.parameters():
            param.requiresgrad = False
```



```

elif strategy == 'finetune':
    # 解冻layer4和fc
    for name, param in model.namedparameters():
        if "layer4" not in name and "fc" not in name:
            param.requiresgrad = False

    model.fc = nn.Linear(model.fc.infeatures, numclasses)
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    model = model.to(device)

    # 3. 损失函数和优化器
    criterion = nn.CrossEntropyLoss()

    # 使用标签平滑 (Label Smoothing) 防止过拟合
    criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

    optimizer = torch.optim.AdamW(
        filter(lambda p: p.requiresgrad, model.parameters),
        lr=learningrate,
        weight_decay=0.05
    )

    # 学习率调度器: Cosine Annealing
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, Tmax=epochs, eta_min=1e-6
    )

    # 4. 训练循环
    best_val_acc = 0.0
    for epoch in range(epochs):
        # 训练阶段
        model.train()
        train_loss = 0.0
        train_correct = 0
        train_total = 0

```

```

for images, labels in trainloader:
    images, labels = images.to(device), labels.to(device)

    optimizer.zero_grad()
    outputs = model(images)
    loss = criterion(outputs, labels)
    loss.backward()

    # 梯度裁剪
    torch.nn.utils.clip_grad_norm(model.parameters(), max_norm)

    optimizer.step()

    trainloss += loss.item()
    , predicted = outputs.max(1)
    traintotal += labels.size(0)
    traincorrect += predicted.eq(labels).sum().item()

# 验证阶段
model.eval()
valloss = 0.0
valcorrect = 0
valtotal = 0

with torch.no_grad():
    for images, labels in valloader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)

        valloss += loss.item()
        , predicted = outputs.max(1)
        valtotal += labels.size(0)
        valcorrect += predicted.eq(labels).sum().item()

valacc = 100.0 * valcorrect / valtotal

```

```

# 保存最佳模型
if val_acc > best_val_acc:
    best_val_acc = val_acc
    torch.save(model.state_dict(), 'best_model.pth')

scheduler.step()

print(f'Epoch {epoch+1}/{epochs} ',
      f'Train Loss: {train_loss/len(train_loader):.4f} ',
      f'Train Acc: {100*train_correct/train_total:.2f}% ',
      f'Val Acc: {val_acc:.2f}% ',
      f'LR: {scheduler.get_last_lr()[0]:.6f}')

return model, best_val_acc

```

2.4 经典模型详细架构

LeNet-5 (1998) ——开山之作

LeNet-5 是 Yann LeCun 设计的第一个成功的卷积神经网络，用于手写数字识别。

输入： $32 \times 32 \times 1$ （灰度图）

- C1： 卷积 5×5 ， 6个滤波器 → $28 \times 28 \times 6$
- S2： 平均池化 2×2 ， 步长2 → $14 \times 14 \times 6$
- C3： 卷积 5×5 ， 16个滤波器 → $10 \times 10 \times 16$
- S4： 平均池化 2×2 ， 步长2 → $5 \times 5 \times 16$
- C5： 卷积 5×5 ， 120个滤波器 → $1 \times 1 \times 120$
- F6： 全连接 84个神经元
- 输出： 10个类别

参数量： ~60K

特点：

- 交替使用卷积和池化（这一模式延续至今）
- 使用 Sigmoid / Tanh 激活（当时 ReLU 还未提出）
- 参数量小，适合 CPU 训练

AlexNet (2012) ——深度学习引爆者

AlexNet 在 ImageNet 竞赛中将 Top-5 错误率从 26% 降到 16%，引爆了深度学习。

输入： $224 \times 224 \times 3$

→ Conv1: 11×11 , 96 滤波器, 步长 4 → $55 \times 55 \times 96$ + ReLU + MaxPool

→ Conv2: 5×5 , 256 滤波器 → $27 \times 27 \times 256$ + ReLU + MaxPool

→ Conv3: 3×3 , 384 滤波器 → $13 \times 13 \times 384$ + ReLU

→ Conv4: 3×3 , 384 滤波器 → $13 \times 13 \times 384$ + ReLU

→ Conv5: 3×3 , 256 滤波器 → $13 \times 13 \times 256$ + ReLU + MaxPool

→ FC6: 4096 + ReLU + Dropout(0.5)

→ FC7: 4096 + ReLU + Dropout(0.5)

→ FC8: 1000 (ImageNet 类别数)

参数量： ~ 60M

关键创新：

1. ReLU 激活函数： 解决 Sigmoid 的梯度消失问题
 $f(x) = \max(0, x)$ ，计算简单，梯度为 1（正区间）
2. Dropout： 随机丢弃 50% 的神经元，防止过拟合
3. 数据增强： 随机裁剪、水平翻转、颜色扰动
4. GPU 训练： 使用两块 GTX 580 并行训练
5. 局部响应归一化 (LRN)： 后来的研究表明这一层不太重要

VGGNet (2014) ——深度的力量

VGGNet 的核心贡献：用多个 3×3 小卷积核替代大卷积核，在减少参数的同时增加网络深度。

为什么 3×3 卷积核堆叠等价于大卷积核？

两个 3×3 卷积堆叠 = 一个 5×5 感受野

三个 3×3 卷积堆叠 = 一个 7×7 感受野

但参数更少：

- 一个 7×7 卷积： $7 \times 7 \times C \times C = 49C^2$
- 三个 3×3 卷积： $3 \times (3 \times 3 \times C \times C) = 27C^2$

而且多了非线性层（每个 3×3 卷积后都有 ReLU）

VGG-16 结构：

输入： $224 \times 224 \times 3$

$\rightarrow [Conv3-64] \times 2 \rightarrow 224 \times 224 \times 64 \rightarrow MaxPool \rightarrow 112 \times 112 \times 64$
 $\rightarrow [Conv3-128] \times 2 \rightarrow 112 \times 112 \times 128 \rightarrow MaxPool \rightarrow 56 \times 56 \times 128$
 $\rightarrow [Conv3-256] \times 3 \rightarrow 56 \times 56 \times 256 \rightarrow MaxPool \rightarrow 28 \times 28 \times 256$
 $\rightarrow [Conv3-512] \times 3 \rightarrow 28 \times 28 \times 512 \rightarrow MaxPool \rightarrow 14 \times 14 \times 512$
 $\rightarrow [Conv3-512] \times 3 \rightarrow 14 \times 14 \times 512 \rightarrow MaxPool \rightarrow 7 \times 7 \times 512$
 $\rightarrow FC: 4096 \rightarrow 4096 \rightarrow 1000$

参数量： $\sim 138M$

特点： 结构规整，所有卷积都是 3×3 ，所有池化都是 2×2

缺点： 参数量大，全连接层占了90%的参数

GoogLeNet / Inception (2014) ——多尺度特征

GoogLeNet 的核心创新是 Inception

模块：在同一个层中并行使用不同大小的卷积核，捕获不同尺度的特征。

Inception 模块 v1：

输入特征图

$\begin{matrix} / & \backslash & \backslash \\ 1 \times 1 & 1 \times 1 & 1 \times 1 \end{matrix}$

$MaxPool$

$\begin{matrix} 3 \times 3 & 5 \times 5 & 1 \times 1 \end{matrix}$

$\backslash \quad / \quad /$

Filter Concat

关键设计：

1. 先用 1×1 卷积降维（减少通道数），再用 3×3 / 5×5 卷积

这叫“瓶颈设计” (Bottleneck)

例如： 256 通道 $\rightarrow 1 \times 1 \times 64 \rightarrow 3 \times 3 \times 64 \rightarrow 3 \times 3 \times 256$

参数量： $64 \times 256 \times 1 \times 1 + 64 \times 64 \times 3 \times 3 + 256 \times 64 \times 3 \times 3 = 16K +$

不用瓶颈： 256 通道 $\rightarrow 3 \times 3 \times 256 = 256 \times 256 \times 3 \times 3 \approx 590K$

2. 1×1 卷积的作用：

- 降维：减少通道数，降低计算量
- 升维：增加非线性表达能力
- 跨通道信息融合

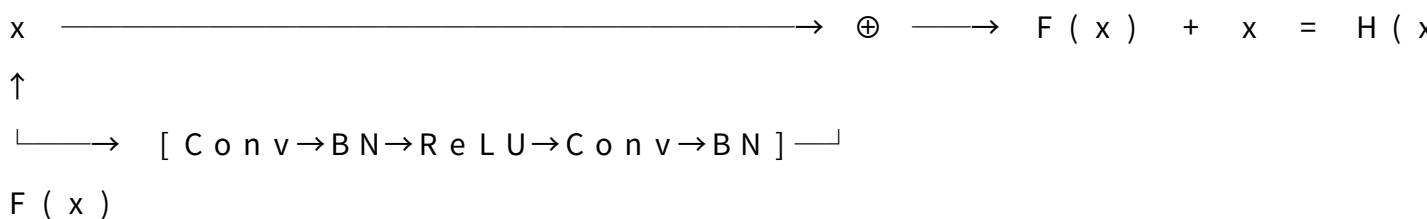
参数量： $\sim 6.8M$ （比VGG少20倍！）

ResNet（2015）——残差连接的革命

ResNet

解决了一个关键问题：网络越深，训练越难（退化问题：56层比20层效果差，不是因为过拟合，而是因为优化困难）

残差块（Residual Block）：



核心公式： $H(x) = F(x) + x$

为什么有效？

1. 如果恒等映射是最优的， $F(x)$ 只需学0，比学 $H(x) = x$ 更容易
2. 梯度可以直接通过跳跃连接回传，缓解梯度消失
3. $F(x)$ 学习的是残差（残差 = 目标 - 输入），更容易优化

数学推导：

前向传播： $y_l = F(x_l) + x_l$

反向传播： $\partial L / \partial x_l = \partial L / \partial y_l \times (1 + \partial F / \partial x_l)$

$\uparrow$

这一项 ≥ 1 ，梯度不会消失！

ResNet 残差块的实现

```
import torch
```

```
import torch.nn as nn
```

```
class BasicBlock(nn.Module):
```

```
""" ResNet 基础残差块（用于ResNet - 18 / 34） """
```

```
expansion = 1
```

```
def __init__(self, inchannels, outchannels, stride=1,
             super().__init__())
```

```
self.conv1 = nn.Conv2d(inchannels, outchannels, 3,
```

```
self.bn1 = nn.BatchNorm2d(outchannels)
```

```
self.relu = nn.ReLU(inplace=True)
```

```
self.conv2 = nn.Conv2d(outchannels, outchannels, 3,
```

```
self.bn2 = nn.BatchNorm2d(outchannels)
```

```
self.downsample = downsample # 降采样（维度不匹配时使用 $1 \times 1$ 卷积调
```

```
def forward(self, x):
```

```
identity = x
```

```
out = self.conv1(x)
```

```
out = self.bn1(out)
```

```
out = self.relu(out)
```

```
out = self.conv2(out)
```

```
out = self.bn2(out)
```

```
if self.downsample is not None:
```

```
identity = self.downsample(x) # 调整维度以匹配
```

```
out += identity # 残差连接！
```

```
out = self.relu(out)
```

```
return out
```

```
class Bottleneck(nn.Module):
```

```
""" ResNet 瓶颈残差块（用于ResNet - 50 / 101 / 152） """
```

```
expansion = 4
```

```
def __init__(self, inchannels, outchannels, stride=1,
             super().__init__())
```

```
#  $1 \times 1$  降维
```

```

self.conv1 = nn.Conv2d(inchannels, outchannels, 1)
self.bn1 = nn.BatchNorm2d(outchannels)
# 3×3 卷积
self.conv2 = nn.Conv2d(outchannels, outchannels, 3)
self.bn2 = nn.BatchNorm2d(outchannels)
# 1×1 升维
self.conv3 = nn.Conv2d(outchannels, outchannels, 1)
self.bn3 = nn.BatchNorm2d(outchannels)
self.relu = nn.ReLU(inplace=True)
self.downsample = downsample

def forward(self, x):
    identity = x

    out = self.relu(self.bn1(self.conv1(x))) # 1×1 降维
    out = self.relu(self.bn2(self.conv2(out))) # 3×3 卷积
    out = self.bn3(self.conv3(out)) # 1×1 升维

    if self.downsample is not None:
        identity = self.downsample(x)

    out += identity # 残差连接
    out = self.relu(out)

    return out

```

ResNet-50 的完整结构

```

def resnet50():
    return nn.Sequential(
        # Stem
        nn.Conv2d(3, 64, 7, 2, 3, bias=False), # 224→112
        nn.BatchNorm2d(64),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(3, 2, 1), # 112→56

        # Stage 1: 3个Bottleneck, 64→256

```



```
# Stage 2: 4个Bottleneck, 128→512
# Stage 3: 6个Bottleneck, 256→1024
# Stage 4: 3个Bottleneck, 512→2048

# Global Average Pooling
nn.AdaptiveAvgPool2d((1, 1)),
nn.Flatten(),
nn.Linear(2048, 1000)
)
```

2.5 ViT (Vision Transformer) 详解

ViT 由 Dosovitskiy et al. 2020 提出，核心思想：将图像切分为 Patch 序列，像处理文本序列一样用 Transformer 处理图像。

ViT 工作流程：

1. 图像切分：

$224 \times 224 \times 3$ 图像 $\rightarrow$ 切分为 16×16 的 Patch $\rightarrow 14 \times 14 = 196$ 个 Patch

2. Patch Embedding：

每个 Patch ($16 \times 16 \times 3 = 768$) $\rightarrow$ 线性映射 $\rightarrow$ 768 维向量

3. 位置编码：

为每个 Patch 位置添加可学习的位置编码

4. [CLS] Token：

在序列开头添加一个特殊的分类 Token

5. Transformer Encoder：

多头自注意力 + MLP，重复 L 次

6. 分类：

取 [CLS] Token 的输出 $\rightarrow$ MLP $\rightarrow$ 类别

数学表达：

$$x_{patch} = [x^{p^1 \cdot E}; x^{p^2 \cdot E}; \dots; x^{p^N \cdot E}] + E_{pos} \# P$$

$$z_0 = [x_{class}; x_{patch}] \# \text{添加 CLS token}$$

```

z' l = MSA ( LN ( z { l - 1 } ) ) + z { l - 1 } # 多头自注意力+残差
z l = MLP ( LN ( z ' l ) ) + z ' l # MLP+残差
y = LN ( z L ^ 0 ) # 取CLS token输出

```

ViT 的简化实现

```

class PatchEmbedding ( nn . Module ) :
    """ 将图像切分为Patch并嵌入 """

    def __init__( self , imgsize = 224 , patchsize = 16 , inchannel = 3 ,
        super () . init ()

    self . numpatches = ( imgsize // patchsize ) * 2 # 196

    # 用卷积实现Patch嵌入 (比reshape+线性更高效)
    self . proj = nn . Conv2d (
        inchannel , embeddim ,
        kernelsize = patchsize , stride = patchsize
    )

    def forward ( self , x ) :
        # x : ( B , 3 , 224 , 224 )
        x = self . proj ( x ) # ( B , 768 , 14 , 14 )
        x = x . flatten ( 2 ) # ( B , 768 , 196 )
        x = x . transpose ( 1 , 2 ) # ( B , 196 , 768 )
        return x

class VisionTransformer ( nn . Module ) :
    """ Vision Transformer 简化实现 """

    def __init__( self , imgsize = 224 , patchsize = 16 , inchannel = 3 ,
        numclasses = 1000 , embeddim = 768 , depth = 12 ,
        numheads = 12 , mlpratio = 4.0 , dropout = 0.1 ) :
        super () . init ()

    # Patch嵌入
    self . patchembed = PatchEmbedding ( imgsize , patchsize , inchannel ,
        numpatches = self . patchembed . numpatches

```

```

# CLS Token
self.clstoken = nn.Parameter(torch.zeros(1, 1, embeddim))

# 位置编码
self.posembed = nn.Parameter(torch.zeros(1, num_patches, embeddim))
self.posdrop = nn.Dropout(dropout)

# Transformer Encoder
encoderlayer = nn.TransformerEncoderLayer(
    dmodel=embeddim,
    nhead=numheads,
    dimfeedforward=int(embeddim * mlpratio),
    dropout=dropout,
    activation='gelu',
    batchfirst=True,
    normfirst=True # Pre-LN (更稳定的训练方式)
)
self.encoder = nn.TransformerEncoder(encoderlayer, num_layers)

# 分类头
self.norm = nn.LayerNorm(embeddim)
self.head = nn.Linear(embeddim, num_classes)

def forward(self, x):
    B = x.shape[0]

    # 1. Patch嵌入
    x = self.patchembed(x) # (B, 196, 768)

    # 2. 添加CLS Token
    clstokens = self.clstoken.expand(B, -1, -1)
    x = torch.cat([clstokens, x], dim=1) # (B, 197, 768)

    # 3. 添加位置编码
    x = x + self.posembed

```

```

x = self.posdrop(x)

# 4. Transformer 编码
x = self.encoder(x)

# 5. 取CLS Token进行分类
x = self.norm(x[:, 0]) # (B, 768)
x = self.head(x) # (B, numclasses)

return x

```

ViT 的关键特性

特性 说明

数据饥渴 小数据集上表现不如CNN（缺乏归纳偏置），需要大规模预训练

全局感受野 自注意力天然具有全局感受野，CNN需要深层才能获得

计算复杂度 $O(N^2)$ 的注意力计算，N为Patch数（图像分辨率高时开销大）

归纳偏置少 CNN有平移不变性和局部性，ViT几乎不假设数据结构

2.6 CLIP——零样本分类的革命

CLIP (Contrastive Language - Image Pre-training) 由 OpenAI 于 2021 年提出，核心思想：通过对比学习，将图像和文本映射到同一语义空间，实现零样本分类。

CLIP 训练过程：

图文对：(Image1, "a photo of a dog"), (Image2, "a photo of a cat")

Image Encoder Text Encoder

↓ ↓

Image1 → [fimg(I1)] ←——对比学习——→ [ftext(T1)] ← "a photo of a dog"

Image2 → [fimg(I2)] ←——对比学习——→ [ftext(T2)] ← "a photo of a cat"

目标：让匹配的图文对相似度高，不匹配的相似度低

Loss = CrossEntropy(similaritymatrix) + CrossEntropy(similaritymatrix)

零样本分类：

对于未知类别，构造提示模板： " a photo of a {classname} "

例如： [" a photo of a cat " , " a photo of a dog " , " a p

将图像编码后与所有文本编码计算相似度，取最高的作为预测类别

CLIP 零样本分类示例

```
import torch
from PIL import Image
from transformers import CLIPModel, CLIPProcessor

model = CLIPModel.from_pretrained("openai/clip-vit
processor = CLIPProcessor.from_pretrained("openai/

def zeroshotclassify(imagepath: str, classnames:
    """CLIP 零样本分类"""

    image = Image.open(imagepath)

    # 构造文本提示
    textprompts = [f"a photo of a {name}" for name in

    # 预处理
    inputs = processor(
        text=textprompts, images=image,
        return_tensors="pt", padding=True
    )

    # 前向推理
    with torch.no_grad():
        outputs = model(inputs)

    # 计算图文相似度
    logits = outputs.logits_per_image # (1, numclasses)
    probs = logits.softmax(dim=1).squeeze()

    # 输出结果
    results = {}
    for name, prob in zip(classnames, probs):
```

```

results[name] = f"{prob.item():.2%}"

return results

```

使用示例：无需训练即可分类任意类别

```

result = zeroshotclassify(
    "testimage.jpg",
    ["cat", "dog", "bird", "fish", "horse"]
)
print(result)

```

输出： { 'cat': '85.32%', 'dog': '8.15%', 'bird': '3.15%', 'fish': '2.15%', 'horse': '1.15%' }

- - -

三、物体检测详解

3.1 两阶段检测器的详细演进

两阶段检测器的核心思路：先找出可能存在目标的区域（Region Proposal），再对每个区域进行分类和精修*。

R-CNN（2014）——开创者

R-CNN 流程：

1. Selective Search：从图像中生成~2000个候选区域
2. Warp：将每个候选区域缩放到固定大小（ 224×224 ）
3. CNN Feature：用CNN提取每个区域的特征（4096维）
4. SVM分类：对每个区域用SVM判断类别
5. BBox Regression：回归修正边界框位置

缺点：

- 速度极慢：每张图需要2000次CNN前向传播 → 一张图47秒
- 训练复杂：CNN、SVM、回归器需要分别训练
- 存储浪费：特征需要保存到磁盘

Fast R-CNN（2015）——端到端训练

F a s t R - C N N 改进：

- 1 . 整图通过C N N： 只需一次C N N前向传播，得到特征图
- 2 . R O I P o o l i n g： 从特征图上直接裁剪候选区域（无需W a r p）
- 3 . 全连接分类： 替换S V M，用全连接层直接分类 + 回归

R O I P o o l i n g 的关键作用：

- 不同大小的候选区域 → P o o l i n g到固定大小（如 7×7 ）
- 在特征图上操作，而非原图，效率大幅提升

速度提升： 一张图约0.3秒（快了150倍）

仍存在的问题： S e l e c t i v e S e a r c h仍是瓶颈（C P U操作，约2秒）

F a s t e r R - C N N（2015）——真正端到端

F a s t e r R - C N N 核心创新： R P N（R e g i o n P r o p o s a l N e t w o r k）

用神经网络替代S e l e c t i v e S e a r c h生成候选区域！

R P N 工作流程：

- 1 . 在C N N特征图上，每个位置放置9个锚框（3尺度 $\times$ 3比例）
- 2 . 对每个锚框预测：
 - 是否包含目标（o b j e c t n e s s s c o r e）
 - 边界框修正量（4个值： d x , d y , d w , d h）
- 3 . 筛选T o p - N候选区域送入R O I P o o l i n g

锚框设计：

- 3个尺度： 128^2 , 256^2 , 512^2
- 3个比例： $1:1$, $1:2$, $2:1$
- 每个位置9个锚框，总共约20000个

锚框修正公式：

$$\begin{aligned}x &= x_{\text{anchor}} + dx \times w_{\text{anchor}} \\y &= y_{\text{anchor}} + dy \times h_{\text{anchor}} \\w &= w_{\text{anchor}} \times \exp(dw) \\h &= h_{\text{anchor}} \times \exp(dh)\end{aligned}$$

速度： 一张图约0.2秒（GPU），实时性仍不如单阶段检测器

精度： 长期以来是精度最高的检测框架

Faster R-CNN 使用示例

```
import torchvision
from torchvision.models.detection import fasterrcnnresnet50fpnv2
```

加载预训练模型

```
model = fasterrcnnresnet50fpnv2(pretrained=True)
model.eval()
```

推理

```
import torch
from PIL import Image
from torchvision import transforms
```

```
def detectobjects(imagepath, confidencethreshold=0.5):
    """使用 Faster R-CNN 检测物体"""
    image = Image.open(imagepath).convert("RGB")
    transform = transforms.Compose([transforms.ToTensor()])
    imgtensor = transform(image)
```

```
with torch.no_grad():
    predictions = model([imgtensor])
```

解析结果

```
boxes = predictions[0]['boxes']
scores = predictions[0]['scores']
labels = predictions[0]['labels']
```

过滤低置信度的检测

```
mask = scores > confidencethreshold
results = []
for box, score, label in zip(boxes[mask], scores[mask], labels[mask]):
    results.append({
        'bbox': box.tolist(), # [x1, y1, x2, y2]
```



```
'score': score.item(),  
'labelid': label.item()  
})  
  
return results
```

3.2 单阶段检测器详解

YOLO 系列的架构演进

YOLOv1 (2016): You Only Look Once

核心思想：将检测问题转化为回归问题

- 将图像分为 $S \times S$ 网格 (7×7)
- 每个网格预测 B 个边界框 (2个) 和 C 个类别概率 (20类)
- 输出: $S \times S \times (5B + C) = 7 \times 7 \times 30$
- 速度: 45 FPS (实时检测的开创者)
- 缺点: 对小目标和密集目标检测效果差

YOLOv3 (2018):

- Darknet-53 主干网络
- 多尺度检测: 在3个不同尺度上检测
- 锚框机制: 每个尺度3个锚框
- 类别预测: 用独立逻辑分类器替代 Softmax (支持多标签)

YOLOv5 (2020): 工业首选

- Ultralytics 实现 (非官方命名但广泛使用)
- CSPNet 主干 + PANet 特征融合
- 多种尺寸: n / s / m / l / x (速度 - 精度权衡)
- 丰富的数据增强: Mosaic, MixUp, HSV增强
- 自动锚框聚类

YOLOv8 (2023): 最新架构

- Anchor-free: 不再使用锚框, 直接预测中心点和宽高
- C2f 模块: 替代 CSP 模块, 更好的梯度流
- Decoupled Head: 分类和回归解耦
- DFL (Distribution Focal Loss): 更精确的边界框回归

YOLOv8 使用示例

```
from ultralytics import YOLO
```

加载模型

```
model = YOLO('yolov8n.pt') # nano版本, 最快
```

训练

```
results = model.train(
    data='coco128.yaml',
    epochs=100,
    imgsz=640,
    batch=16,
    lr0=0.01,
    lrf=0.01, # 最终学习率 = lr0 * lrf
    augment=True,
    mosaic=1.0, # Mosaic数据增强概率
    mixup=0.1, # MixUp概率
)
```

推理

```
results = model('testimage.jpg', conf=0.5)
```

可视化

```
for result in results:
    boxes = result.boxes
    for box in boxes:
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        conf = box.conf[0].item()
        cls = box.cls[0].item()
        print(f"检测到 {model.names[int(cls)]}, 置信度 {conf:.2f} "
              f"({x1:.0f}, {y1:.0f}) - ({x2:.0f}, {y2:.0f})")

# 保存带标注的图片
result.save('output.jpg')
```

3.3 关键技术详解

锚框 (Anchor Box) 设计

K-means 锚框聚类

```
import numpy as np
from sklearn.cluster import KMeans

def anchorclustering(annotations, numanchors=9):
    """ 使用K-means对训练集的边界框进行聚类,生成锚框 """

    # 提取所有边界框的宽高
    widths = []
    heights = []
    for ann in annotations:
        w = ann['bbox'][2] # 宽度
        h = ann['bbox'][3] # 高度
        widths.append(w)
        heights.append(h)

    # 归一化到[0, 1]
    wh = np.array(list(zip(widths, heights)))
    wh = wh / wh.max(axis=0)

    # K-means 聚类
    kmeans = KMeans(nclusters=numanchors, randomstate=0)
    kmeans.fit(wh)

    # 获取聚类中心作为锚框
    anchors = kmeans.clustercenters
    anchors = anchors * wh.max(axis=0) # 还原到原始尺度
    anchors = sorted(anchors, key=lambda x: x[0] * x[1])

    return anchors
```

IoU 感知的聚类 (更好的方法)

```
def iouawareclustering(annotations, numanchors=9)
```

" " " 使用 I o U 距离的 K - m e a n s 聚类 " " "

```
def ioudistance(box, centroid):  
    " " " 计算 1 - I o U 作为距离度量 " " "  
    # 计算交集  
    inter = min(box[0], centroid[0]) min(box[1], centroid[1])  
    # 计算并集  
    union = box[0] box[1] + centroid[0] centroid[1] - inter  
    return 1 - inter / union  
  
# . . . 实现类似 K - m e a n s 但用 I o U 距离  
pass
```

NMS（非极大值抑制）详解

```
def nms(boxes, scores, iouthreshold=0.5):  
    " " "
```

非极大值抑制（Non-Maximum Suppression）

算法步骤：

- 1 . 按置信度从高到低排序
- 2 . 选择最高分的框，加入结果
- 3 . 计算该框与所有剩余框的 I o U
- 4 . 移除 I o U > 阈值的框（认为它们检测的是同一个目标）
- 5 . 重复 2 - 4 直到所有框都被处理

A r g s :

boxes: (N, 4) 边界框 [x1, y1, x2, y2]

scores: (N,) 置信度分数

iouthreshold: I o U 阈值

" " "

```
import numpy as np
```

```
x1 = boxes[:, 0]
```

```
y1 = boxes[:, 1]
```

```
x2 = boxes[:, 2]
```

```

y2 = boxes[:, 3]

areas = (x2 - x1) * (y2 - y1) # 计算面积
order = scores.argsort()[::-1] # 按分数降序排列

keep = []
while order.size > 0:
    i = order[0]
    keep.append(i)

    # 计算当前框与所有剩余框的IoU
    xx1 = np.maximum(x1[i], x1[order[1:]])
    yy1 = np.maximum(y1[i], y1[order[1:]])
    xx2 = np.minimum(x2[i], x2[order[1:]])
    yy2 = np.minimum(y2[i], y2[order[1:]])

    inter = np.maximum(0, xx2 - xx1) * np.maximum(0, yy2 - yy1)
    iou = inter / (areas[i] + areas[order[1:]] - inter)

    # 保留IoU小于阈值的框
    inds = np.where(iou <= iouthreshold)[0]
    order = order[inds + 1]

return keep

```

Soft-NMS：更温和的抑制方式

```

def softnms(boxes, scores, iouthreshold=0.5, sigma=0.5):
    """

```

Soft-NMS：不是直接删除重叠框，而是降低其分数

对于IoU高的框，分数按高斯衰减：

```

score = score * exp(-iou^2 / sigma)

```

优点：当两个目标确实重叠时，不会误删

```

"""

```

```

import numpy as np

```

```

x1 = boxes[:, 0]

```

```

y1 = boxes[:, 1]
x2 = boxes[:, 2]
y2 = boxes[:, 3]
areas = (x2 - x1) * (y2 - y1)

order = scores.argsort()[::-1]
keep = []

while order.size > 0:
    i = order[0]
    keep.append(i)

    xx1 = np.maximum(x1[i], x1[order[1:]])
    yy1 = np.maximum(y1[i], y1[order[1:]])
    xx2 = np.minimum(x2[i], x2[order[1:]])
    yy2 = np.minimum(y2[i], y2[order[1:]])

    inter = np.maximum(0, xx2 - xx1) * np.maximum(0, yy2 - yy1)
    iou = inter / (areas[i] + areas[order[1:]] - inter)

    # Soft-NMS: 高斯衰减分数
    scores[order[1:]] = np.exp(-(iou ** 2) / sigma)

    # 移除分数过低的框
    inds = np.where(scores[order[1:]] > 0.01)[0]
    order = order[inds + 1]

return keep

```

FPN (特征金字塔网络) 详解

FPN 核心思想： 融合多尺度特征，解决目标多尺度问题

自底向上路径 (Bottom-Up)： 正常的CNN前向传播，特征图逐渐缩小

自顶向下路径 (Top-Down)： 从最深层开始，逐步上采样

横向连接 (Lateral Connection)： 1×1 卷积对齐通道数后相加

$P5$ ($1/32$ 尺度) $\longrightarrow$ $P5_{out}$
 $\uparrow$ 上采样 $2\times$
 $P4$ ($1/16$ 尺度) $\longrightarrow$ 1×1 卷积 $\longrightarrow$ 相加 $\longrightarrow$ $P4_{out}$
 $\uparrow$ 上采样 $2\times$
 $P3$ ($1/8$ 尺度) $\longrightarrow$ 1×1 卷积 $\longrightarrow$ 相加 $\longrightarrow$ $P3_{out}$
 $\uparrow$
 $P2$ ($1/4$ 尺度) $\longrightarrow$ 1×1 卷积 $\longrightarrow$ 相加 $\longrightarrow$ $P2_{out}$

每层输出后接 3×3 卷积消除混叠效应

大目标在深层 ($P5$) 检测, 小目标在浅层 ($P2 / P3$) 检测

3.4 DETR——端到端检测

DETR (Detection Transformer) 由 Facebook 于 2020 年提出, 核心创新: 用 Transformer 替代锚框和 NMS, 实现真正的端到端检测。

DETR 架构:

1. CNN Backbone: 提取特征图
2. Transformer Encoder: 增强特征表示
3. Transformer Decoder:
 - 输入 N 个可学习的 Object Query
 - 每个 Query 对应一个潜在目标
 - 输出 N 个预测 (包含 "无目标" 类)
4. Hungarian Matching: 二部图匹配, 将预测与 GT 配对
5. Set Loss: 计算匹配后的损失

关键优势:

- 无需锚框设计
- 无需 NMS 后处理
- 全局推理能力

关键挑战:

- 训练收敛慢 (需要 500 epochs)
- 小目标检测不如 Faster R-CNN
- 推理速度不如 YOLO

3.5 评估指标详解

IoU (交并比)

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

```
def calculate_iou(box1, box2):
```

```
    """ 计算两个边界框的 IoU
```

```
    Args:
```

```
    box1, box2: [x1, y1, x2, y2] 格式的边界框
```

```
    Returns:
```

```
    IoU值, 范围 [0, 1]
```

```
    """
```

```
    # 计算交集
```

```
    x1 = max(box1[0], box2[0])
```

```
    y1 = max(box1[1], box2[1])
```

```
    x2 = min(box1[2], box2[2])
```

```
    y2 = min(box1[3], box2[3])
```

```
    interarea = max(0, x2 - x1) * max(0, y2 - y1)
```

```
    # 计算并集
```

```
    box1area = (box1[2] - box1[0]) * (box1[3] - box1[1])
```

```
    box2area = (box2[2] - box2[0]) * (box2[3] - box2[1])
```

```
    unionarea = box1area + box2area - interarea
```

```
    iou = interarea / unionarea if unionarea > 0 else 0
```

```
    return iou
```

mAP (平均精度均值)

mAP 计算步骤:

1. 对每个类别:

- a . 按置信度从高到低排序所有检测
- b . 逐个判断是否为 T P (I o U > 阈值且未匹配过) 或 F P
- c . 计算每个 r e c a l l 点对应的 p r e c i s i o n
- d . 绘制 P r e c i s i o n - R e c a l l 曲线
- e . 计算 A P = 曲线下面积 (通常用 1 1 点插值或全点插值)

2 . m A P = 所有类别 A P 的平均值

常用标准 :

- m A P @ 0 . 5 : I o U 阈值 = 0 . 5 时的 m A P (较宽松)
- m A P @ 0 . 5 : 0 . 9 5 : I o U 阈值从 0 . 5 到 0 . 9 5 (步长 0 . 0 5) 的平均 m A P (更严格)
- m A P @ 0 . 7 5 : I o U 阈值 = 0 . 7 5 时的 m A P (较严格)

实际数值参考 (C O C O 数据集) :

- Y O L O v 5 s : m A P @ 0 . 5 = 5 6 . 8 % , m A P @ 0 . 5 : 0 . 9 5 = 3 7 . 4 %
- Y O L O v 8 x : m A P @ 0 . 5 = 6 7 . 0 % , m A P @ 0 . 5 : 0 . 9 5 = 4 6 . 6 %
- F a s t e r R - C N N : m A P @ 0 . 5 : 0 . 9 5 = 4 2 . 0 % (R e s N e t - 5 0 - F P)

```
def calculateap(recalls, precisions):
    """计算AP (Average Precision)
```

使用全点插值法 (A l l - P o i n t I n t e r p o l a t i o n)

"""

在每个 r e c a l l 点, 取该点及之后最大的 p r e c i s i o n

这样 P R 曲线是单调递减的

a p = 0 . 0

f o r i i n r a n g e (1 , l e n (r e c a l l s)) :

a p += (r e c a l l s [i] - r e c a l l s [i - 1]) p r e c i s i o n s [i]

r e t u r n a p

```
def calculatemap(alldetections, allgroundtruths,
    """计算mAP
```

A r g s :

a l l d e t e c t i o n s : 所有检测结果, 按类别组织

a l l g r o u n d t r u t h s : 所有真值标注

```

iouthreshold: IoU阈值
"""

aps = []
for classid in range(numclasses):
    # 获取该类别的所有检测和真值
    detections = alldetections[classid]
    groundtruths = allgroundtruths[classid]

    # 按置信度排序
    detections.sort(key=lambda x: x['score'], reverse=True)

    # 计算TP / FP
    tp = np.zeros(len(detections))
    fp = np.zeros(len(detections))

    matchedgts = set()

    for i, det in enumerate(detections):
        bestiou = 0
        bestgtidx = -1

        for j, gt in enumerate(groundtruths):
            iou = calculateiou(det['bbox'], gt['bbox'])
            if iou > bestiou:
                bestiou = iou
                bestgtidx = j

        if bestiou >= iouthreshold and bestgtidx not in matchedgts:
            tp[i] = 1
            matchedgts.add(bestgtidx)
        else:
            fp[i] = 1

    # 计算precision和recall
    tpcumsum = np.cumsum(tp)
    fpcumsum = np.cumsum(fp)

```

```

precisions = tpcumsum / (tpcumsum + fpcumsum)
recalls = tpcumsum / len(groundtruths)

# 计算AP
ap = calculateap(recalls, precisions)
aps.append(ap)

return np.mean(aps) # mAP

- - -

```

四、语义分割详解

4.1 FCN（全卷积网络）——开创者

FCN 由 Long et al. 2015 提出，核心创新：将分类网络中的全连接层替换为卷积层，

FCN 的关键设计：

1. 全卷积化：

分类网络： $\text{Conv} \rightarrow \text{Conv} \rightarrow \dots \rightarrow \text{FC} \rightarrow \text{FC} \rightarrow \text{Softmax}$ （1个类别概率）

FCN： $\text{Conv} \rightarrow \text{Conv} \rightarrow \dots \rightarrow \text{Conv } 1 \times 1 \rightarrow \text{Conv}$ （像素级类别概率）

2. 转置卷积（反卷积）：

普通卷积：大特征图 $\rightarrow$ 小特征图（下采样）

转置卷积：小特征图 $\rightarrow$ 大特征图（上采样）

转置卷积的计算过程：

输入 $2 \times 2 \rightarrow$ 填充 $\rightarrow 6 \times 6 \rightarrow$ 卷积 3×3 ，步长1 $\rightarrow$ 输出 4×4

数学表达： $y = C^T x$ ，其中C是卷积矩阵

3. 跳跃连接（Skip Connection）：

FCN-32s：直接从1/32尺度上采样32倍 $\rightarrow$ 粗糙

FCN-16s：融合1/16和1/32的特征 $\rightarrow$ 较好

FCN-8s：融合1/8、1/16和1/32的特征 $\rightarrow$ 最好

越浅层的特征空间分辨率越高，细节越丰富

越深层的特征语义信息越强，定位越准

FCN 的简化实现

```
class FCN(nn.Module):
    def __init__(self, numclasses=21):
        super().__init__()

        # 使用VGG16作为backbone
        vgg = models.vgg16(pretrained=True)

        # 编码器（特征提取）
        self.features = vgg.features # 到pool5为止

        # 1×1卷积替代全连接层
        self.conv6 = nn.Conv2d(512, 4096, 7)
        self.conv7 = nn.Conv2d(4096, 4096, 1)
        self.score = nn.Conv2d(4096, numclasses, 1)

        # 转置卷积（上采样）
        self.upscore2 = nn.ConvTranspose2d(numclasses, numclasses, 2, 2)
        self.upscore8 = nn.ConvTranspose2d(numclasses, numclasses, 8, 8)

        # 跳跃连接的1×1卷积
        self.scorepool4 = nn.Conv2d(512, numclasses, 1)
        self.scorepool3 = nn.Conv2d(256, numclasses, 1)

    def forward(self, x):
        # 编码器
        features = self.features(x)

        # 分类得分
        x = F.relu(self.conv6(features))
        x = F.dropout(x, 0.5, self.training)
        x = F.relu(self.conv7(x))
        x = F.dropout(x, 0.5, self.training)
        x = self.score(x)
```

```

# 上采样
x = self.upscore2(x) # 2倍上采样
# ... 跳跃连接融合
x = self.upscore8(x) # 8倍上采样到原始尺寸

return x

```

4.2 U-Net——医学影像的标配

U-Net 由 Ronneberger et al. 2015 提出，最初用于医学图像分割，核心特 + 跳跃连接。

U-Net 结构（以 512×512 输入为例）：

编码器（左半部分，收缩路径）：

$512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32$ （尺寸）
 $1 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$ （通道数）

解码器（右半部分，扩展路径）：

$32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32$ （尺寸，通过转置卷积上采样）
 （通道数，通过拼接和卷积减少）

跳跃连接：将编码器对应层的特征图拼接到解码器

作用：保留浅层的高分辨率空间细节信息

为什么 U-Net 在医学影像上效果特别好？

1. 医学影像目标边界精确：跳跃连接保留了空间细节
2. 训练数据少：数据增强（弹性变形）+ 结构先验
3. 目标形态多变：多尺度特征融合

U-Net 实现示例

```

class UNet(nn.Module):
    def __init__(self, in_channels=1, num_classes=2):
        super().__init__()

# 编码器
self.enc1 = self.convblock(in_channels, 64)

```

```

self.enc2 = self.convblock(64, 128)
self.enc3 = self.convblock(128, 256)
self.enc4 = self.convblock(256, 512)

# 瓶颈层
self.bottleneck = self.convblock(512, 1024)

# 解码器
self.upconv4 = nn.ConvTranspose2d(1024, 512, 2, 2)
self.dec4 = self.convblock(1024, 512) # 1024=512

self.upconv3 = nn.ConvTranspose2d(512, 256, 2, 2)
self.dec3 = self.convblock(512, 256)

self.upconv2 = nn.ConvTranspose2d(256, 128, 2, 2)
self.dec2 = self.convblock(256, 128)

self.upconv1 = nn.ConvTranspose2d(128, 64, 2, 2)
self.dec1 = self.convblock(128, 64)

# 输出层
self.out = nn.Conv2d(64, numclasses, 1)

self.pool = nn.MaxPool2d(2)

def convblock(self, inch, outch):
    """双卷积块：Conv→BN→ReLU→Conv→BN→ReLU"""
    return nn.Sequential(
        nn.Conv2d(inch, outch, 3, padding=1),
        nn.BatchNorm2d(outch),
        nn.ReLU(inplace=True),
        nn.Conv2d(outch, outch, 3, padding=1),
        nn.BatchNorm2d(outch),
        nn.ReLU(inplace=True),
    )

def forward(self, x):

```

编码器

```
e1 = self.enc1(x) # (B, 64, H, W)
```

```
e2 = self.enc2(self.pool(e1)) # (B, 128, H/2, W/2)
```

```
e3 = self.enc3(self.pool(e2)) # (B, 256, H/4, W/4)
```

```
e4 = self.enc4(self.pool(e3)) # (B, 512, H/8, W/8)
```

瓶颈层

```
b = self.bottleneck(self.pool(e4)) # (B, 1024, H/8, W/8)
```

解码器 + 跳跃连接

```
d4 = self.upconv4(b) # (B, 512, H/8, W/8)
```

```
d4 = torch.cat([d4, e4], dim=1) # 拼接跳跃连接
```

```
d4 = self.dec4(d4) # (B, 512, H/8, W/8)
```

```
d3 = self.upconv3(d4)
```

```
d3 = torch.cat([d3, e3], dim=1)
```

```
d3 = self.dec3(d3)
```

```
d2 = self.upconv2(d3)
```

```
d2 = torch.cat([d2, e2], dim=1)
```

```
d2 = self.dec2(d2)
```

```
d1 = self.upconv1(d2)
```

```
d1 = torch.cat([d1, e1], dim=1)
```

```
d1 = self.dec1(d1)
```

```
return self.out(d1)
```

4.3 DeepLab 系列详解

DeepLab 演进路线：

DeepLabv1 (2015)：

- 核心创新： 空洞卷积 (Atrous / Dilated Convolution)
- 问题： 普通卷积 + 池化会降低分辨率
- 解决： 空洞卷积在不增加参数 / 计算量的情况下扩大感受野

普通 3×3 卷积：感受野 = 3×3

空洞率 = 2 的 3×3 卷积：感受野 = 5×5 (参数量不变!)

空洞率 = 4 的 3×3 卷积：感受野 = 9×9

空洞卷积的权重只在特定位置非零：

```
rate=1:  1  0  0   rate=2:  1  0  1  0  1
0  0  0  0  0  0  0  0
0  0  1  1  0  1  0  1
...  0  0  0  0  0
1  0  1  0  1
```

- CRF 后处理：精修边界

DeepLabv2 (2016)：

- ASPP (Atrous Spatial Pyramid Pooling)：

并行使用不同空洞率的卷积 (1, 6, 12, 18)，融合多尺度信息

DeepLabv3 (2017)：

- 改进 ASPP：空洞率 1, 6, 12, 18 + 全局平均池化

- 去掉 CRF 后处理

DeepLabv3+ (2018)：

- 编码器 - 解码器结构：

编码器：Xception + ASPP

解码器：简单的上采样模块 + 低级特征融合

- 成为主流语义分割框架

4.4 分割损失函数详解

===== Dice Loss =====

解决类别不平衡问题 (前景像素远少于背景像素)

Dice 系数 = $2 |A \cap B| / (|A| + |B|)$ ，衡量两个集合的重叠度

```
class DiceLoss(nn.Module):
    def __init__(self, smooth=1.0):
        super().__init__()
```



```

self.smooth = smooth # 防止除以0

def forward(self, predictions, targets):
    """
    Args:
    predictions: (B, C, H, W) 模型输出 (经过Softmax)
    targets: (B, H, W) 真值标签
    """
    # 将targets转为one-hot
    numclasses = predictions.shape[1]
    targetsoneshot = F.onehot(targets, numclasses).perm(0, 2, 3, 1)

    # 计算每个类别的Dice系数
    intersection = (predictions * targetsoneshot).sum(dim=(2, 3))
    union = predictions.sum(dim=(2, 3)) + targetsoneshot.sum(dim=(2, 3))

    dice = (2.0 * intersection + self.smooth) / (union + self.smooth)
    diceloss = 1.0 - dice.mean()

    return diceloss

```

===== Focal Loss for Segmentation =====

降低简单样本的权重，让模型更关注难样本

```

class FocalLoss(nn.Module):
    def __init__(self, alpha=0.25, gamma=2.0):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma

    def forward(self, predictions, targets):
        """计算Focal Loss"""
        celoss = F.crossentropy(predictions, targets, reduction='none')
        pt = torch.exp(-celoss) # 预测正确类别的概率

        # Focal权重: (1 - pt) ^ gamma
        # 当pt大 (简单样本) 时, 权重小

```

```

# 当p t小（难样本）时，权重大
focalweight = (1 - pt) * self.gamma

# Alpha平衡
alphaweight = self.alpha * (1 - targets.float()) + 1

loss = alphaweight * focalweight * ce_loss
return loss.mean()

===== 组合损失 =====
class CombinedLoss(nn.Module):
    """ 组合多种损失函数 """

    def __init__(self, ceweight=1.0, diceweight=1.0, focalweight=1.0):
        super().__init__()
        self.ce_weight = ceweight
        self.dice_weight = diceweight
        self.focal_weight = focalweight

        self.ce_loss = nn.CrossEntropyLoss()
        self.dice_loss = DiceLoss()
        self.focal_loss = FocalLoss()

    def forward(self, predictions, targets):
        ce = self.ce_loss(predictions, targets)
        dice = self.dice_loss(F.softmax(predictions, dim=-1), targets)
        focal = self.focal_loss(predictions, targets)

        total = self.ce_weight * ce + self.dice_weight * dice + self.focal_weight * focal
        return total

- - -

```

五、实例分割详解

5.1 Mask R-CNN 架构详解

Mask R-CNN 在 Faster R-CNN 基础上增加了一个分割分支，同时输出边界框、类别

Mask R-CNN 架构：

Faster R-CNN：

图像 $\rightarrow$ Backbone $\rightarrow$ RPN $\rightarrow$ ROI Pooling $\rightarrow$ [分类头，回归头]

Mask R-CNN：

图像 $\rightarrow$ Backbone $\rightarrow$ RPN $\rightarrow$ ROI Align $\rightarrow$ [分类头，回归头，分割头]

↓

[类别] [边界框] [$K \times m \times m$ 掩码]

ROI Align vs ROI Pooling：

ROI Pooling 的问题：两次量化导致像素偏移

1. 将浮点坐标量化到网格点（第一次量化）
2. 在每个网格内再次量化（第二次量化）

结果：对于小目标，偏移量可达几十个像素

ROI Align 的解决：使用双线性插值，不做任何量化

1. 将ROI分成 $k \times k$ 个bin
2. 在每个bin内采样4个点
3. 对每个采样点使用双线性插值计算特征值
4. 对bin内所有采样点取平均

数学表达：

对于采样点 (x, y) ，其特征值：

$$f(x, y) = \sum G(i, j) \times \max(0, 1 - x - i) \times \max(0, 1 -$$

其中 $G(i, j)$ 是特征图上最近邻的4个点的值

Mask R-CNN 使用示例

```
import torchvision
from torchvision.models.detection import maskrcnn
```

加载预训练模型

```
model = maskrcnnresnet50fpnv2(pretrained=True)
```

```

model.eval()

def instancesegment(imagepath, confidence=0.7):
    """实例分割"""
    image = Image.open(imagepath).convert("RGB")
    imgtensor = transforms.ToTensor()(image)

    with torch.no_grad():
        predictions = model([imgtensor])

    pred = predictions[0]
    masks = pred['masks'] # (N, 1, H, W) 每个实例的掩码
    boxes = pred['boxes'] # (N, 4) 边界框
    labels = pred['labels'] # (N,) 类别
    scores = pred['scores'] # (N,) 置信度

    # 过滤低置信度
    mask = scores > confidence

    results = []
    for i in range(mask.sum()):
        instance = {
            'mask': masks[i, 0].numpy(), # 二值掩码
            'box': boxes[i].tolist(),
            'label': labels[i].item(),
            'score': scores[i].item()
        }
        results.append(instance)

    return results

```

5.2 全景分割 (Panoptic Segmentation)

全景分割 = 语义分割 + 实例分割的统一

全景分割的目标：

对图像中每个像素分配一个类别ID和实例ID

- `stuff` 类别（不可数，如天空、道路）： 只需语义标签
- `thing` 类别（可数，如人、车）： 需要实例标签

评估指标： `PQ` (`Panoptic Quality`)

$$PQ = SQ \times RQ$$

`SQ` (`Segmentation Quality`)： 分割质量（匹配实例的 `IoU` 均值）

`RQ` (`Recognition Quality`)： 识别质量 ($TP / (TP + FP + FN)$)

- - -

六、其他重要视觉任务

6.1 目标跟踪

目标跟踪： 在视频的连续帧中持续定位目标

`Single Object Tracking` (`SOT`)：

- 第一帧给定目标位置，后续帧持续跟踪
- 核心挑战： 外观变化、遮挡、尺度变化、快速运动

代表方法：

- `SiamFC` (2016)： 孪生网络，模板匹配
- `SiamRPN` (2018)： 加入区域提议网络
- `OSTrack` (2022)： `Transformer-based tracker`
- `MixFormer` (2022)： 混合注意力机制

6.2 姿态估计

人体姿态估计： 从图像中检测人体关键点

2D姿态估计： 输出每个关键点的 (`x` , `y`) 坐标

关键点： 头、肩、肘、腕、髁、膝、踝（17个关键点）

代表方法：

- `OpenPose` (2017)： 自底向上，`Part Affinity Fields`
- `HRNet` (2019)： 高分辨率网络保持空间精度
- `MediaPipe`： `Google` 的实时姿态估计

3D姿态估计： 输出每个关键点的 (x , y , z) 坐标

额外挑战： 深度信息恢复、遮挡处理

6.3 3D视觉新进展

NeRF (Neural Radiance Fields, 2020):

- 用MLP隐式表示3D场景
- 输入： 3D坐标 (x , y , z) + 观察方向 (θ , ϕ)
- 输出： 颜色 (r , g , b) + 密度 (σ)
- 体渲染： 从任意视角生成2D图像
- 缺点： 训练慢（需要数小时）、无法实时渲染

3D Gaussian Splatting (2023):

- 用3D高斯椭球显式表示场景
- 每个高斯有： 位置 μ 、协方差 Σ 、颜色 c 、不透明度 α
- 可微光栅化： 实时渲染 (> 100 FPS)
- 优点： 训练快（分钟级）、渲染快、质量高
- 已成为3D重建和渲染的新SOTA

NeRF 简化概念代码

```
class NeRFModel(nn.Module):
```

```
    """NeRF 的核心 MLP"""
```

```
    def __init__(self, posdim=63, dirdim=27, hiddendim=256):
        super().__init__()
```

```
        # 位置编码（高频编码）
```

```
        #  $\gamma(p) = [\sin(2^0 \pi p), \cos(2^0 \pi p), \dots, \sin(2^L \pi p), \cos(2^L \pi p)]$ 
```

```
        #  $L=10$ , 维度 =  $3 \times 2 \times 10 = 60$  (加上原始3维 = 63)
```

```
        # 密度网络
```

```
        self.densitynet = nn.Sequential(
            nn.Linear(posdim, hiddendim), nn.ReLU(),
            nn.Linear(hiddendim, hiddendim), nn.ReLU(),
            nn.Linear(hiddendim, hiddendim), nn.ReLU(),
```

```

nn.Linear(hiddendim, hiddendim), nn.ReLU(),
nn.Linear(hiddendim, 1), # 输出密度 $\sigma$ 
)

# 颜色网络（条件于观察方向）
self.colonet = nn.Sequential(
nn.Linear(hiddendim + dirdim, hiddendim // 2), nn.ReLU(),
nn.Linear(hiddendim // 2, 3), nn.Sigmoid(), # 输出RGB
)

def forward(self, positions, directions):
"""
Args:
positions: (N, 63) 位置编码后的3D坐标
directions: (N, 27) 方向编码后的观察方向
"""
sigma = self.densitynet(positions) # (N, 1)
rgb = self.colonet(torch.cat([positions, directions]))
return rgb, sigma

- - -

```

七、CV 模型训练技巧和 Tricks

7.1 训练技巧汇总

技巧 说明 适用阶段

Warmup 学习率从0线性增加到设定值 训练初期

Cosine Annealing 学习率按余弦曲线衰减 训练全程

Label Smoothing 软化标签（0.9而非1.0） 分类任务

Mixup / CutMix 数据混合增强 数据加载

EMA 参数的指数移动平均 训练全程

梯度裁剪 限制梯度范数 训练全程

梯度累积 小batch模拟大batch 显存不足时

知识蒸馏 大模型指导小模型 模型压缩

测试时增强 (T T A) 多种增强推理取平均 推理阶段

混合精度训练 F P 1 6 + F P 3 2 混合 加速训练

7.2 关键训练技巧实现

==== EMA (Exponential Moving Average) =====

```
class ModelEMA:
```

```
    """ 模型参数的指数移动平均 """
```

```
    def __init__(self, model, decay=0.9999):
```

```
        self.decay = decay
```

```
        self.shadow = {} # EMA参数
```

```
        self.backup = {} # 用于验证时临时切换
```

```
    # 初始化EMA参数
```

```
    for name, param in model.named_parameters():
```

```
        if param.requires_grad:
```

```
            self.shadow[name] = param.data.clone()
```

```
    def update(self, model):
```

```
        """ 更新EMA参数 """
```

```
    for name, param in model.named_parameters():
```

```
        if param.requires_grad:
```

```
            newaverage = (1.0 - self.decay) param.data + self
```

```
            self.shadow[name] = newaverage.clone()
```

```
    def apply_shadow(self, model):
```

```
        """ 将EMA参数应用到模型（用于验证 / 推理） """
```

```
    for name, param in model.named_parameters():
```

```
        if param.requires_grad:
```

```
            self.backup[name] = param.data
```

```
            param.data = self.shadow[name]
```

```
    def restore(self, model):
```

```
        """ 恢复原始参数 """
```

```
    for name, param in model.named_parameters():
```



```

if param.requires_grad:
    param.data = self.backup[name]
self.backup = {}

===== TTA (Test-Time Augmentation) =====
def ttapredict(model, image, transformslist):
    """测试时增强预测"""
    predictions = []

    for transform in transformslist:
        # 应用增强
        augmented = transform(image)
        # 预测
        pred = model(augmented)
        # 反向增强 (如翻转回来)
        pred = reversetransform(pred, transform)
        predictions.append(pred)

    # 取平均
    return torch.mean(torch.stack(predictions), dim=0)

===== 混合精度训练 =====
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()

for images, labels in trainloader:
    optimizer.zero_grad()

    with autocast(): # 自动混合精度
        outputs = model(images)
        loss = criterion(outputs, labels)

    scaler.scale(loss).backward() # 缩放梯度
    scaler.step(optimizer) # 更新参数
    scaler.update() # 更新缩放因子

```

- - -

八、数据标注工具和流程

8.1 常用标注工具

工具 支持任务 特点

LabelImg 目标检测 经典，简单易用

Labelme 分割、检测 支持多边形标注

CVAT 全任务 Intel开源，功能强大

Label Studio 全任务 支持多模态标注

Roboflow 全任务 在线平台，集成训练

Labelbox 全任务 企业级标注平台

VGG Image Annotator 分割 轻量级Web工具

8.2 标注流程最佳实践

1. 制定标注规范

- 类别定义和示例
- 边界框标注规则（紧密 / 宽松？）
- 遮挡处理规则
- 模糊 / 不确定情况的处理

2. 试标注与一致性检查

- 多人标注同一样本
- 计算标注者间一致性（IoU > 0.8为合格）
- 根据差异修订标注规范

3. 批量标注

- 使用预标注模型加速（主动学习）
- 质量抽查（10%的样本双人标注）

4. 质量控制

- 自动检查：边界框面积、长宽比、类别分布
- 人工抽检

5 . 数据增强和后处理

- 清洗异常标注
- 格式转换（COCO、VOC、YOLO等）

- - -

九、C V 项目完整 workflow

C V 项目完整 workflow
1 . 需求分析
├─ 任务类型确定（分类 / 检测 / 分割）
├─ 精度和速度要求
└─ 部署环境（云端 / 边缘 / 移动端）
2 . 数据准备
├─ 数据收集（公开数据集 + 自采数据）
├─ 数据标注（选择工具、制定规范、质量检查）
├─ 数据增强策略
└─ 数据集划分（训练 / 验证 / 测试，通常 7 : 1 : 2）
3 . 模型选择
├─ 基于任务选择模型架构
├─ 考虑预训练权重和迁移学习
└─ 速度 - 精度权衡
4 . 模型训练
├─ 超参数搜索（学习率、batch size、epoch等）
├─ 训练技巧（数据增强、EMA、混合精度等）
├─ 监控训练过程（loss曲线、验证指标）
└─ 调优迭代
5 . 模型评估

	└─ 在测试集上评估	
	└─ 错误分析（哪类样本分错了？为什么？）	
	└─ 鲁棒性测试（不同光照、角度、遮挡条件）	
6 .	模型部署	
	└─ 模型导出（ONNX、TensorRT、TFLite）	
	└─ 推理优化（量化、剪枝、蒸馏）	
	└─ 后端服务（FastAPI、Triton）	
	└─ 监控和迭代	
7 .	持续改进	
	└─ 在线数据收集（困难样本挖掘）	
	└─ 模型更新（增量学习）	
	└─ A / B 测试	

典型项目配置

- 智能安防：检测（YOLOv8）+ 分类（ResNet）+ 跟踪（ByteTrack）多模型融合
- 工业质检：语义分割（U-Net）定位缺陷区域 + 分类（EfficientNet）判断缺陷类型
- 自动驾驶：检测（DETR / Faster R-CNN）+ 分割（DeepLabv3+）+ 关键点检测
- 医学影像：分割（U-Net++ / nnU-Net）+ 分类（DenseNet）+ 配准
- 移动端：MobileNetV3 / YOLOv8n + 量化 + TFLite部署
- - -

十、选型指南

场景 推荐方案 理由

资源受限 轻量级分类模型（MobileNetV3）或单阶段检测器（YOLOv8n） 参数量小、推理快

实时性要求 TensorRT加速的检测模型（YOLOv8+TensorRT可达150FPS） 优化推理性能

精细分析 语义分割→场景解析；实例分割→商品计数 像素级精度

数据标注成本 分类最低 → 检测中等 → 实例分割最高 按需选择

零样本场景 CLIP零样本分类 无需训练数据

小数据集 迁移学习 + 强数据增强 利用预训练知识

- - -

十一、未来发展方向

- 1 . Transformer架构重塑：Swin Transformer统一检测与分割特征提取，Ma
- 2 . 多模态学习：CLIP推动分类任务向零样本学习发展，LLM+CV实现视觉对话
- 3 . 3D感知技术：NeRF → 3D Gaussian Splatting，延伸分割任务到空间维
- 4 . 模型轻量化：RepVG结构、知识蒸馏、量化（INT8 / INT4）、剪枝
- 5 . 自动化机器学习：NAS自动搜索最优架构，AutoML降低使用门槛
- 6 . 边缘计算部署：端侧实时推理，NPU / TPU加速
- 7 . 生成式CV：Stable Diffusion、DALL-E，从理解到创造
- 8 . 世界模型：视觉+行动的统一建模，视频理解和预测

- - -

核心总结：理解四大任务的核心差异与内在联系，是构建高效计算机视觉系统的关键。从分类到实例分割的技术演进，不仅反映了算法能力的提升，更体现了对真实世界感知需求的深度理解。现代CV项目的成功，不仅取决于模型选择，更取决于数据质量、训练技巧和工程部署能力的综合运用。